



35. Ridge & Lasso Regression

"Let's leave out the unnecessary stuff — no need to carry extra weight."

Previous Section:

 [34. Rule-Based Classifiers](#)

Contents:

[1. Linear Regression – Revisited](#)

[2. Lasso and Ridge Regression – Introducing Regularization and the Penalty Term](#)

[3. Lasso and Ridge Regression – Handful Calculations](#)

[Summary](#)

[References](#)

Every model you learn in Machine Learning has a "next version". Not necessarily a replacement, but an evolution. Something that looks almost the same on the surface, yet quietly fixes a problem you didn't even realize was there.

Linear Regression is one of those models. Simple. Elegant. Almost too perfect. You give it data, it draws a line. You ask it to predict, it responds with confidence. And for a while, everything feels under control.

But then reality steps in. The data isn't clean anymore. Features start overlapping. Some variables matter a lot. Others barely matter at all, yet they still sneak into the model.

And suddenly, that perfect line? It starts behaving strangely. It bends too much. It reacts too strongly. It tries to fit everything, even the noise.

This is where the real question appears:

| What if the model is learning too much? → What if "more fitting" is actually worse?

Because remember—Machine Learning is not about memorizing the data. It's about understanding it. And when a model becomes too sensitive to the training data... it loses that understanding. It stops generalizing... and starts overfitting.

So instead of asking: "How do we make the model fit better?" → We shift the question entirely:

| How do we make the model behave better? How do we control what it learns, and what it ignores?

That's exactly where Ridge and Lasso Regression come in. They don't change the core idea of Linear Regression. They don't replace it. They *discipline* it.

They introduce a simple but powerful concept: Not all complexity is good. Not all features deserve equal importance. And most importantly, a good model is not the one that fits everything. It's the one that knows what to ignore.

In this section, we'll revisit Linear Regression—not to relearn it, but to understand its limitations. Because only when you see where something breaks, can you truly understand why the next version exists.

1. Linear Regression – Revisited

Let's go back to something familiar. But this time, look at it a little deeper. When we first learned Linear Regression, everything revolved around one simple idea: *Find the line that fits the data best.*

But what does "best" actually mean? It means minimizing the error between what the model predicts... and what actually exists in reality.

More precisely, Linear Regression tries to minimize something called the **Sum of Squared Errors (SSE)**:

$$SSE = \sum_{i=1}^M (y_i - \hat{y}_i)^2$$

At first glance, this might look like just another formula, but the idea behind it is very simple: Take the difference between the real value and the predicted value. Square it (so errors don't cancel each other out). Add everything together. And then make that total error as small as possible.

That's it. That's the entire philosophy of Linear Regression.

So naturally, the model adjusts itself—changing its coefficients, tilting the line, shifting it up or down—until the predictions $\hat{y}_i$ get as close as possible to the true values y_i .

And we've already seen how this happens: Sometimes through a direct, clean solution (the **Normal Equation**) → Sometimes through step-by-step learning (like **Gradient Descent**). Different paths, same destination.

But now let's step out of that perfect world. Because things start to change when the data becomes more complicated. Imagine this: You don't just have one or two features anymore. You have dozens, maybe hundreds. And some of them start saying the same thing in slightly different ways

This is where trouble begins. The model, trying to minimize error at all costs, starts to *overreact*. It stretches. It adjusts too much. It tries to fit every tiny fluctuation in the data. Even the noise. And that leads us to a very important problem: **Overfitting**

This is when the model becomes *too good* at learning the training data but fails when it sees something new. It doesn't understand the pattern anymore. It just remembers what it saw.

So now the goal changes. We are no longer asking: "How do we minimize error as much as possible?". Instead, we ask something smarter: "*How do we control the model while minimizing error?*"

Because sometimes, a slightly worse fit on training data leads to a much better model in the real world. And this is where a new idea enters: **Regularization**. Instead of only minimizing the error, we *modify the objective itself*.

We say: "**Yes, fit the data...**" → "**But don't go crazy doing it.**"

We introduce a penalty—something that discourages the model from assigning overly large importance (coefficients) to features. In other words, the model is no longer free to do anything it wants → It now has to behave

And from this idea, two powerful variations of Linear Regression emerge: **Ridge Regression and Lasso Regression**.

They don't replace Linear Regression. They refine it. They take the same foundation and add control, balance, and discipline. And that small change? It makes all the difference.

2. Lasso and Ridge Regression – Introducing Regularization and the Penalty Term

So far, Linear Regression has been doing one job: Fit the data as closely as possible. But now we've seen the problem: Sometimes, it fits *too* well. It starts chasing noise. It starts overreacting to patterns that don't really matter. And that's where we step in and say:

■ "**You're allowed to fit the data...**" → "But you're not allowed to go out of control."

That idea is called **Regularization**.

The Shift in Thinking - Originally, our objective looked like this:

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Pure error minimization. Nothing else. But now, we change the rules. We don't just minimize error anymore. We *add a constraint*. A penalty for being too complex. A punishment for large coefficients

So the new objective becomes:

$$\min_{\beta} (\text{Error Term} + \text{Penalty Term})$$

This is the heart of both **Ridge** and **Lasso Regression**. Same idea, different ways of enforcing discipline.

Ridge Regression (L2) – “Control, but don’t eliminate”

Ridge Regression adds a penalty based on the **square of the coefficients**:

$$L_{\text{Ridge}}(\beta) = \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^m \beta_j^2$$

Let’s break that down:

- First term: same old error (SSE)
- Second term: penalty on coefficient size.
- And here’s the key: Large coefficients get punished *much more heavily* → Because they are squared.

What does Ridge actually do? It doesn’t remove features. It doesn’t “turn anything off.” → Instead: It *shrinks* everything, smoothly, proportionally, under control. So every feature still contributes, just less aggressively.

When is Ridge useful? Ridge shines when features are highly correlated (multicollinearity). You believe *all features matter*. You want stability over sensitivity. And mathematically, it even fixes numerical issues:

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$$

That small addition λ makes the system stable and ensures the matrix is invertible

Lasso Regression (L1) – “Know what to ignore”

Now here’s where things get interesting. Lasso uses a **different kind of penalty**:

$$L_{\text{Lasso}}(\beta) = \sum_{i=1}^n (y_i - x_i^T \beta)^2 + \lambda \sum_{j=1}^m |\beta_j|$$

No squares. Just absolute values. And that small change, changes everything.

What does Lasso actually do? Instead of just shrinking coefficients, it can push them **exactly to zero**. Which means: Some features completely disappear from the model. Not reduced, not weakened. **Removed**.

Why does this matter? Because Lasso doesn’t just control the model. It *simplifies* it. It performs **automatic feature selection**. As a result: Fewer features → Cleaner model → Easier interpretation.

When is Lasso useful? Lasso is preferred when many features are irrelevant. You want a simpler, more interpretable model. You need built-in dimensionality reduction

The Role of λ – The “Discipline Knob”

Both Ridge and Lasso are controlled by one parameter: λ (sometimes written as α)

And this parameter decides how strict the model becomes:

- $\lambda = 0$ → Back to standard Linear Regression
- Small λ → Light control
- Large λ → Strong shrinkage
- $\lambda \rightarrow \infty$ → Coefficients approach zero

So in a way: λ controls the balance between fitting the data and controlling complexity. And choosing it correctly? That’s where techniques like **cross-validation** come in.

Ridge vs Lasso – The Real Difference

Now we can finally answer the key question: “*They both regularize... so what’s the difference?*” It all comes down to how they penalize:

Aspect	Ridge (L2)	Lasso (L1)
Penalty	Squared coefficients	Absolute coefficients
Feature Selection	No	Yes
Behavior	Shrinks smoothly	Produces sparse models
Best Use	Correlated features	Many irrelevant features

Regularization changes something fundamental: We are no longer chasing perfect accuracy on training data. Instead, we accept a little bias to gain a lot of stability. Because in Machine Learning: A model that fits *everything*, learns nothing.

So the trade-off becomes:

- Linear Regression → Low bias and high variance (risk of overfitting);
- Ridge / Lasso → Slightly higher bias, much lower variance but better generalization

And that's the real purpose of Ridge and Lasso. They don't try to make the model smarter. They make it *wiser*. Knowing what to learn, and more importantly, knowing what to ignore.

3. Lasso and Ridge Regression – Handful Calculations

Example: The Student First-Semester and Second-Semester score dataset

First-Semester (X)	Second-Semester (Y)	XY
4.5	5.1	22.95
5.2	6.3	32.76
6.0	6.7	40.20
6.4	7.5	48.00
7.1	7.8	55.38
7.3	8.1	59.13
8.0	8.6	68.80
8.4	9.0	75.60
9.1	9.4	85.54
9.5	9.8	93.10

Calculate the required values (they will also be used for Ridge and Lasso Regression):

- $n = 10$
- $\sum(XY) = 581.46$
- $\sum(X) = 71.5$
- $\sum(Y) = 78.3$
- $\sum(X^2) = 535.569$
- $(\sum(X))^2 = 5112.25$

We will train a classic Linear Regression model.

$$\beta_1 = \frac{n \sum(x_i y_i) - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2} = \frac{(10 \cdot 581.46) - (71.5 \cdot 78.3)}{(10 \cdot 535.569) - 5112.25} = 0.8879$$

$$\beta_0 = \frac{\sum y_i - \beta_1 \sum x_i}{n} = \frac{78.3 - (0.8879 \cdot 71.5)}{10} = 1.4815$$

The best fitting line (also called the OLS line):

$$Y = 1.4815 + 0.8879X$$

Prediction on the entire dataset:

First-Semester (X)	Predicted Second-Semester ($\hat{Y}$)	Actual Second-Semester (Y)	Residual
4.5	5.5	5.1	0.4
5.2	6.1	6.3	0.2
6.0	6.8	6.7	0.1
6.4	7.2	7.5	0.3
7.1	7.8	7.8	0
7.3	8.0	8.1	0.1
8.0	8.6	8.6	0
8.4	8.9	9.0	0.1
9.1	9.5	9.4	0.1
9.5	9.9	9.8	0.1

$$SSE = \sum_{i=1}^M (y_i - \hat{y}_i)^2 = 0.34$$

Linear Regression with Python

```
import pandas as pd

sample_data = {'First-Semester Score (X)': [4.5, 5.2, 6.0, 6.4, 7.1, 7.3, 8.0, 8.4, 9.1, 9.5],
               'Second-Semester Score (Y)': [5.1, 6.3, 6.7, 7.5, 7.8, 8.1, 8.6, 9.0, 9.4, 9.8]}

df = pd.DataFrame(sample_data)
```

```
from sklearn.model_selection import train_test_split

# 80% Training, 20% Testing
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)
# Features and Targets
X_train = train_df[['First-Semester Score (X)']]
y_train = train_df['Second-Semester Score (Y)']
X_test = test_df[['First-Semester Score (X)']]
y_test = test_df['Second-Semester Score (Y)']
```

```

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Fit the model
modell = LinearRegression()
modell.fit(X_train, y_train)
# Display the linear regression equation
A = modell.intercept_
B = modell.coef_[0]
print(f"Linear Equation: Y = {A} + ({B} * X)")
# Evaluation
y_predict = modell.predict(X_test)
# Compute the Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_predict)
print(f"Mean Squared Error on the test set: {mse}")
# Compute the R² score
r2 = r2_score(y_test, y_predict)
print(f"R² Score on the test set: {r2}")

```

Linear Equation: Y = 1.1747311827956954 + (0.9301075268817209 * X)

Mean Squared Error on the test set: 0.07016779396462061

R² Score on the test set: 0.9707938422623847

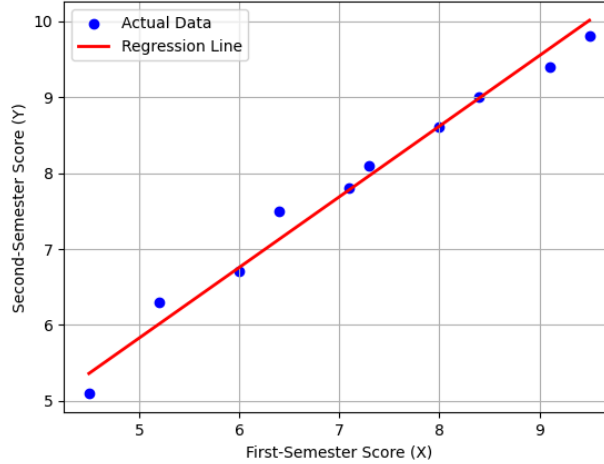
```

import matplotlib.pyplot as plt

# Scatter plot of actual data
plt.scatter(df['First-Semester Score (X)'], df['Second-Semester Score (Y)'],
           color='blue', label='Actual Data')
# Regression line
X_range = [[min(df['First-Semester Score (X)']), [max(df['First-Semester Score (X)'])]]
y_range = modell.predict(X_range)
plt.plot([x[0] for x in X_range], y_range, color='red', linewidth=2,
        label='Regression Line')
# Labels and title
plt.xlabel('First-Semester Score (X)')
plt.ylabel('Second-Semester Score (Y)')
plt.title(f'Linear Regression: First vs Second Semester Scores: Y = {A:.2f} + ({B:.2f} * X)')
plt.legend()
plt.grid(True)
plt.show()

```

Linear Regression: First vs Second Semester Scores: $Y = 1.17 + (0.93 * X)$



Now, let's actually see what **Ridge Regression** does. We already know the Ordinary Least Squares (OLS) solution:

$$\hat{Y}_{OLS} = 1.4815 + 0.8879X$$

Clean. Straightforward. No restrictions. But now, we introduce something new: $\lambda = 10$. A penalty strong enough to *force the model to behave*.

- **Step 1 – Start from the Ridge Objective**

We begin with the Ridge loss:

$$L(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 + \lambda \beta_1^2$$

Notice something subtle—but important: The intercept β_0 is **not penalized**. Only the slope β_1 is controlled. This is exactly how most libraries (like Scikit-Learn) implement Ridge.

- **Step 2 – Center the Problem**

Instead of solving everything directly, we simplify. We introduce two powerful quantities:

$$S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2; \quad S_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

These help us avoid messy expansions and focus directly on what matters.

- **Step 3 – Solve for the Intercept**

Take the derivative with respect to β_0 :

$$\begin{aligned} \frac{\partial L}{\partial \beta_0} &= \frac{\partial}{\partial \beta_0} \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 + \lambda \beta_1^2 \\ &\iff 2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) * \frac{\partial}{\partial \beta_0} (y_i - \beta_0 - \beta_1 x_i) \\ &\iff 2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) * (-1) \\ &\iff (-2) \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) \end{aligned}$$

Set it to zero:

$$\begin{aligned} (-2) \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) &= 0 \\ \iff \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i) &= 0 \end{aligned}$$

Expand:

$$\begin{aligned} \iff \sum_{i=1}^n (y_i) - \sum_{i=1}^n (\beta_0) - \sum_{i=1}^n (\beta_1 x_i) &= 0 \\ \iff \sum_{i=1}^n y_i - n(\beta_0) - \beta_1 \sum_{i=1}^n x_i &= 0 \end{aligned}$$

Rearrange:

$$\begin{aligned} \iff -n(\beta_0) &= -\sum_{i=1}^n (y_i) + \sum_{i=1}^n (\beta_1 x_i) \\ \iff \frac{\sum_{i=1}^n (y_i) - \sum_{i=1}^n (\beta_1 x_i)}{n} & \\ \iff \beta_0 &= \frac{1}{n} \sum_{i=1}^n y_i - \beta_1 \frac{1}{n} \sum_{i=1}^n x_i \end{aligned}$$

So we get the familiar result:

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

This is a key insight: **Ridge does NOT change how the intercept is computed**

- **Step 4 – Solve for the Slope**

Now comes the real difference. Take the derivative with respect to β_1 :

$$\begin{aligned} \frac{\partial L}{\partial \beta_1} &= \frac{\partial}{\partial \beta_1} \left(\sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 + \lambda \beta_1^2 \right) \\ \iff 2 \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)(-x_i) + 2\lambda \beta_1 & \\ \iff (-2) \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) + 2\lambda \beta_1 & \end{aligned}$$

Set it to zero:

$$\begin{aligned} (-2) \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) + 2\lambda \beta_1 &= 0 \\ \iff (-2) \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) &= -2\lambda \beta_1 \\ \iff \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) &= \lambda \beta_1 \end{aligned}$$

Now substitute $\beta_0 = \bar{y} - \beta_1 \bar{x}$:

$$\iff \sum_{i=1}^n x_i (y_i - (\bar{y} - \beta_1 \bar{x}) - \beta_1 x_i) = \lambda \beta_1$$

$$\iff \sum_{i=1}^n x_i (y_i - \bar{y} + \beta_1 \bar{x} - \beta_1 x_i) = \lambda \beta_1$$

$$\iff \sum_{i=1}^n x_i (y_i - \bar{y}) + \beta_1 \bar{x} \sum_{i=1}^n x_i - \beta_1 \sum_{i=1}^n x_i^2 = \lambda \beta_1$$

Since $\sum_{i=1}^n x_i = n\bar{x}$, we can replace the equation as follows:

▼ Prove that $\sum_{i=1}^n x_i = n\bar{x}$?

Suppose a sequence $[1, 2, 3, 4, 5]$

$$\sum_{i=1}^n x_i = 1 + 2 + 3 + 4 + 5 = 15 \iff n\bar{x} = 5 * \frac{15}{5} = 15$$

$$\iff \sum_{i=1}^n x_i (y_i - \bar{y}) - \beta_1 \left(\sum_{i=1}^n x_i^2 - n\bar{x}^2 \right) = \lambda \beta_1$$

$$\iff S_{xy} - \beta_1 S_{xx} = \lambda \beta_1$$

▼ Prove that $\sum_{i=1}^n x_i (y_i - \bar{y}) = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$?

$$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n (x_i y_i - x_i \bar{y} - \bar{x} y_i + \bar{x} \bar{y}) = \sum_{i=1}^n x_i y_i - \bar{y} \sum_{i=1}^n x_i - \bar{x} \sum_{i=1}^n y_i + \sum_{i=1}^n \bar{x} \bar{y} = \sum_{i=1}^n x_i y_i - \bar{y} \sum_{i=1}^n x_i - \bar{x} \sum_{i=1}^n y_i + \sum_{i=1}^n \bar{x} \bar{y}$$

Show middle terms cancel: $\bar{y} \sum_{i=1}^n x_i - \bar{x} \sum_{i=1}^n y_i = 0$

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i, \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

$$\iff \bar{y} \sum_{i=1}^n x_i = \frac{1}{n} \left(\sum_{i=1}^n y_i \right) \left(\sum_{i=1}^n x_i \right), \quad \bar{x} \sum_{i=1}^n y_i = \frac{1}{n} \left(\sum_{i=1}^n x_i \right) \left(\sum_{i=1}^n y_i \right)$$

Continue simplification:

$$\iff \bar{y} \sum_{i=1}^n x_i - \bar{x} \sum_{i=1}^n y_i = 0 \iff \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = \sum_{i=1}^n x_i y_i - \bar{y} \sum_{i=1}^n x_i = \sum_{i=1}^n (x_i y_i - x_i \bar{y}) = \sum_{i=1}^n x_i (y_i - \bar{y})$$

$$\text{Therefore : } \sum_{i=1}^n x_i (y_i - \bar{y}) = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

$$S_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

▼ Prove that $\sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - n\bar{x}^2$?

$$\sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n (x_i^2 - 2x_i \bar{x} + \bar{x}^2) = \sum_{i=1}^n x_i^2 - 2\bar{x} \sum_{i=1}^n x_i + \sum_{i=1}^n \bar{x}^2 = \sum_{i=1}^n x_i^2 - 2\bar{x} \sum_{i=1}^n x_i + n\bar{x}^2$$

Use mean identity:

$$\sum_{i=1}^n x_i = n\bar{x} \Rightarrow -2\bar{x} \sum_{i=1}^n x_i = -2n\bar{x}^2$$

Final simplification:

$$\sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - 2n\bar{x}^2 + n\bar{x}^2 = \sum_{i=1}^n x_i^2 - n\bar{x}^2$$

$$\text{Therefore : } \sum_{i=1}^n (x_i - \bar{x})^2 = \sum_{i=1}^n x_i^2 - n\bar{x}^2$$

$$S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2$$

Rearrange:

$$S_{xy} = \beta_1(S_{xx} + \lambda)$$

And finally:

$$\beta_{1,\text{ridge}} = \frac{S_{xy}}{S_{xx} + \lambda}$$

• Step 5 – Plug in the Numbers

From the dataset:

$$S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2 = 24.345; \quad S_{xy} = \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y}) = 21.615; \quad (\lambda = 10)$$

So:

$$\beta_{1,\text{ridge}} = \frac{21.615}{24.345 + 10} = \frac{21.615}{34.345} = 0.62935$$

Now compute the intercept:

$$\beta_{0,\text{ridge}} = 7.83 - (0.62935 \times 7.15) = 3.33015$$

Final Ridge Model: $\hat{Y}_{\text{ridge}} = 3.33015 + 0.62935X$

Compare with OLS: $\hat{Y}_{OLS} = 1.4815 + 0.8879X$

Now pause, and *look* at what happened. The slope dropped: $0.8879 \rightarrow 0.62935$. The model became **less steep**, less sensitive to changes in X

Prediction by the Ridge model on the entire dataset:

First-Semester ($\mathbf{X}$)	Predicted Second-Semester ($\hat{\mathbf{Y}}$)	Actual Second-Semester ($\mathbf{Y}$)	Residual
4.5	6.2	5.1	1.1
5.2	6.6	6.3	0.3
6.0	7.1	6.7	0.4
6.4	7.4	7.5	0.1
7.1	7.8	7.8	0
7.3	7.9	8.1	0.2
8.0	8.4	8.6	0.2
8.4	8.6	9.0	0.4

9.1	9.1	9.4	0.3
9.5	9.3	9.8	0.5

$$SSE = \sum_{i=1}^M (y_i - \hat{y}_i)^2 = 2.05$$

Ridge didn't change the structure of the model. It changed its **behavior**. It pulled the slope toward zero. It reduced how aggressively the model reacts. It traded a bit of accuracy for stability. And that's the whole point: OLS tries to fit *everything* → Ridge tries to fit but with **discipline**. Because sometimes, a weaker model is actually a *stronger* one in the real world

Ridge Regression with Python

```
# Train the Ridge Regression model (with  $\alpha = 1$ )
from sklearn.linear_model import Ridge

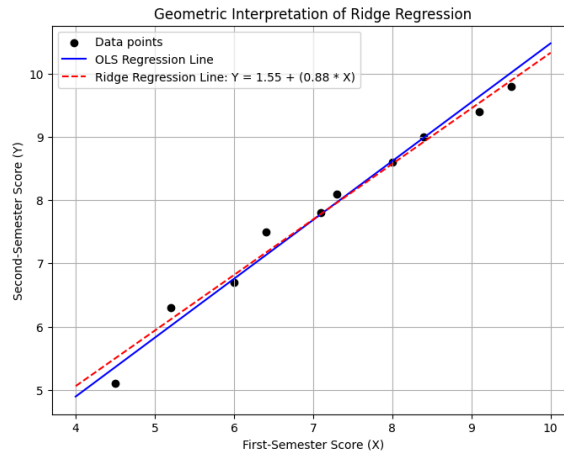
model2 = Ridge(alpha=1)
model2.fit(X_train, y_train)
# Display the Ridge Regression equation
A2 = model2.intercept_
B2 = model2.coef_[0]
print(f"Ridge Equation: Y = {A2} + ({B2} * X)")
# Evaluation
y_predict = model2.predict(X_test)
# Compute the Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_predict)
print(f"Mean Squared Error on the test set: {mse}")
# Compute the R2 score
r2 = r2_score(y_test, y_predict)
print(f"R2 Score on the test set: {r2}")
```

Ridge Equation: Y = 1.549605411499436 + (0.8776775648252537 * X)
Mean Squared Error on the test set: 0.026697965031667374
R² Score on the test set: 0.9888874235039886

```
# Draw both Linear Regression best-fitting line and Ridge Regression line in the same plot.
import matplotlib.pyplot as plt
import numpy as np

x_line = np.linspace(min(df['First-Semester Score (X)'])-0.5, max(df['First-Semester Score (X)'])+0.5, 100)
y_ols = model1.intercept_ + model1.coef_[0] * x_line
y_ridge = model2.intercept_ + model2.coef_[0] * x_line
# Plot
plt.figure(figsize=(8,6))
plt.scatter(df['First-Semester Score (X)'], df['Second-Semester Score (Y)'],
            color="black", label="Data points")
plt.plot(x_line, y_ols, color="blue", label="OLS Regression Line")
plt.plot(x_line, y_ridge, color="red", linestyle="--", label=f"Ridge Regression Line: Y = {A2:.2f} + ({B2:.2f} * X)")
plt.xlabel("First-Semester Score (X)")
plt.ylabel("Second-Semester Score (Y)")
plt.title("Geometric Interpretation of Ridge Regression")
plt.legend()
```

```
plt.grid(True)
plt.show()
```



Now we take the *same dataset*, the *same setup*. But this time, we change one thing: Instead of **Ridge**, we use **Lasso Regression** with the same penalty strength: $\lambda = 10$. And this small change? It will completely change how the model behaves.

- **Step 1 – Start from the Lasso Objective**

The loss function now becomes:

$$L(\beta_0, \beta_1) = \sum_{i=1}^n (y_i - \beta_0 - \beta_1 x_i)^2 + \lambda |\beta_1|$$

Notice the difference: Ridge uses $\beta_1^2 \rightarrow$ Lasso uses $|\beta_1|$. No squares. No smooth curve. Just a sharp, linear penalty.

- **Step 2 – Reuse the Centered Quantities**

From before, we already have: $S_{xx} = 24.345$; $S_{xy} = 21.615 \rightarrow$ These will again become the backbone of our derivation.

- **Step 3 – Solve for the Intercept**

Nothing changes here. The derivative with respect to β_0 gives:

$$\frac{\partial L}{\partial \beta_0} = 0 \Rightarrow \beta_0 = \bar{y} - \beta_1 \bar{x}$$

So again: $\beta_0 = \bar{y} - \beta_1 \bar{x} \rightarrow$ Lasso does **not** affect the intercept.

- **Step 4 – The Key Difference: Derivative of $|\beta_1|$**

Now things get interesting. The derivative of the absolute value is not smooth:

$$\frac{d|\beta_1|}{d\beta_1} = \begin{cases} +1 & \text{if } \beta_1 > 0 \\ -1 & \text{if } \beta_1 < 0 \end{cases}$$

So instead of one equation, we now have **cases**.

- **Step 5 – Derivative with respect to β_1**

Start from:

$$\frac{\partial L}{\partial \beta_1} = (-2) \sum_{i=1}^n x_i (y_i - \beta_0 - \beta_1 x_i) + \lambda \cdot \text{sign}(\beta_1)$$

Using the same simplification as before:

$$\sum_{i=1}^n x_i(y_i - \beta_0 - \beta_1 x_i) = S_{xy} - \beta_1 S_{xx}$$

So the equation becomes:

$$2(S_{xy} - \beta_1 S_{xx}) + \lambda \cdot \text{sign}(\beta_1) = 0$$

- **Step 6 – Case Analysis**

Case 1: $\beta_1 > 0$. Then $\text{sign}(\beta_1) = +1$

$$2(S_{xy} - \beta_1 S_{xx}) + \lambda = 0$$

Expand:

$$2S_{xy} + 2\beta_1 S_{xx} + \lambda = 0$$

Rearrange:

$$2\beta_1 S_{xx} = 2S_{xy} - \lambda \Rightarrow \beta_1 = \frac{2S_{xy} - \lambda}{2S_{xx}}$$

Simplify:

$$\beta_1 = \frac{S_{xy} - \frac{\lambda}{2}}{S_{xx}}$$

Case 2: $\beta_1 < 0$. Then $\text{sign}(\beta_1) = -1$

$$(-2)(S_{xy} - \beta_1 S_{xx}) - \lambda = 0 \iff (-2)S_{xy} + 2\beta_1 S_{xx} - \lambda = 0$$

$$2\beta_1 S_{xx} = 2S_{xy} + \lambda \Rightarrow \beta_1 = \frac{2S_{xy} + \lambda}{2S_{xx}}$$

Simplify:

$$\beta_1 = \frac{S_{xy} + \frac{\lambda}{2}}{S_{xx}}$$

- **Step 7 – Determine the Correct Case**

We check:

$$S_{xy} - \frac{\lambda}{2} = 21.615 - 5 = 16.615 > 0$$

So: $\beta_1 > 0 \rightarrow$ We use **Case 1**

- **Step 8 – Compute the Slope**

$$\beta_1 = \frac{16.615}{24.345} = 0.68248$$

- **Step 9 – Compute the Intercept**

$$\beta_0 = 7.83 - (0.68248 \times 7.15) = 2.950268$$

Final Lasso Model

$$\hat{Y}_{lasso} = 2.950268 + 0.68248X$$

Compare Everything

Model	Intercept	Slope
OLS	1.4815	0.8879
Ridge	3.33015	0.62935
Lasso	2.95027	0.68248

Look closely at the slopes:

- OLS: 0.8879 (no control);
- Ridge: 0.62935 (strong shrinkage);
- Lasso: 0.68248 (selective shrinkage).

Lasso behaves differently. It doesn't just shrink smoothly like Ridge. It **pulls coefficients toward zero in a more aggressive, directional way**

And if λ were larger? That β_1 could become **exactly 0**. Which means the model would say: *"This feature? I don't need it."*

Ridge says: *"Keep everything... but reduce their influence."*; Lasso says: *"If something isn't important... remove it."* → And that difference? That's what turns Lasso from a regularizer into a **feature selector**

Prediction by the Lasso model on the entire dataset:

First-Semester ($\mathbf{X}$)	Predicted Second-Semester ($\hat{\mathbf{Y}}$)	Actual Second-Semester ($\mathbf{Y}$)	Residual
4.5	6.0	5.1	0.9
5.2	6.5	6.3	0.2
6.0	7.0	6.7	0.3
6.4	7.3	7.5	0.2
7.1	7.8	7.8	0
7.3	7.9	8.1	0.2
8.0	8.4	8.6	0.2
8.4	8.7	9.0	0.3
9.1	9.2	9.4	0.2
9.5	9.4	9.8	0.4

$$SSE = \sum_{i=1}^M (y_i - \hat{y}_i)^2 = 1.396778$$

Lasso Regression with Python

```

from sklearn.linear_model import Lasso

model3 = Lasso(alpha=1)
model3.fit(X_train, y_train)
# Display the Lasso Regression equation
A3 = model3.intercept_
B3 = model3.coef_[0]
print(f"Lasso Equation: Y = {A3} + ({B3} * X)")
# Evaluation
y_predict = model2.predict(X_test)
# Compute the Mean Squared Error (MSE)
mse = mean_squared_error(y_test, y_predict)
print(f"Mean Squared Error on the test set: {mse}")
# Compute the R2 score
r2 = r2_score(y_test, y_predict)
print(f"R2 Score on the test set: {r2}")

```

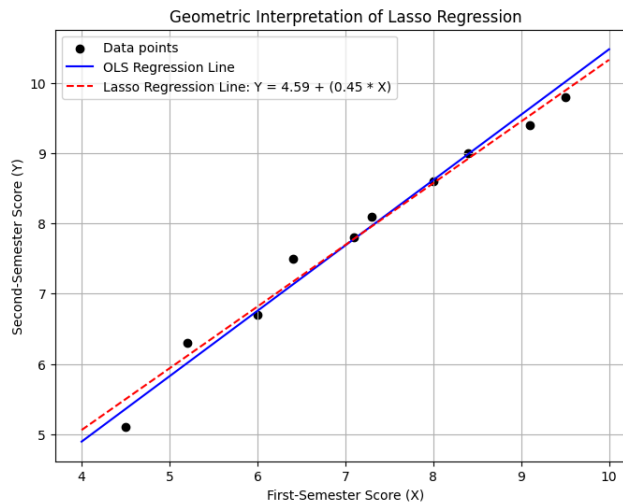
Lasso Equation: Y = 4.591696535244922 + (0.45221027479092 * X)
Mean Squared Error on the test set: 0.026697965031667374
R² Score on the test set: 0.9888874235039886

```

# Draw both Linear Regression best-fitting line and Lasso Regression line in the same plot.
import matplotlib.pyplot as plt
import numpy as np

x_line = np.linspace(min(df['First-Semester Score (X)'])-0.5,
max(df['First-Semester Score (X)')+0.5, 100)
y_ols = model1.intercept_ + model1.coef_[0] * x_line
y_lasso = model2.intercept_ + model2.coef_[0] * x_line
# Plot
plt.figure(figsize=(8,6))
plt.scatter(df['First-Semester Score (X)'], df['Second-Semester Score (Y)'],
color="black", label="Data points")
plt.plot(x_line, y_ols, color="blue", label="OLS Regression Line")
plt.plot(x_line, y_lasso, color="red", linestyle="--", label=f"Lasso Regression Line: Y = {A3:.2f} + ({B3:.2f} * X)")
plt.xlabel("First-Semester Score (X)")
plt.ylabel("Second-Semester Score (Y)")
plt.title("Geometric Interpretation of Lasso Regression")
plt.legend()
plt.grid(True)
plt.show()

```



Summary

Ridge and Lasso Regression are technically different models. The equations change. The optimization changes. Even the solutions we derive are no longer the same as standard Linear Regression.

But at their core? They are not trying to *replace* Linear Regression. They are trying to *fix it*.

- Linear Regression asks one question: *"How do I fit the data as closely as possible?"*
- Ridge and Lasso ask a better one: *"How do I fit the data... without losing control?"*

They introduce a simple but powerful idea: Add a penalty → Control the coefficients → Prevent the model from becoming too sensitive

And from that idea, two different philosophies emerge: **Ridge** shrinks everything... smoothly and continuously; **Lasso** pushes things to the edge and removes what doesn't matter

So the math is different. But conceptually? They are still Linear Regression. Just more disciplined.

And that's the real takeaway: A good model is not the one that fits the data perfectly. It's the one that knows how far it should go and when to stop

References

<https://www.geeksforgeeks.org/machine-learning/ridge-regression-vs-lasso-regression/>

<https://www.datacamp.com/tutorial/tutorial-lasso-ridge-regression>

<https://www.ibm.com/think/topics/ridge-regression>

https://www.youtube.com/watch?v=Xm2C_gTAI8c

<https://www.youtube.com/watch?v=w1X19Md7TQ>

Next Section:

 [36. Decision Tree Regressor](#)